Session 2: Images, Video, Color Spaces, Filtering, Edges

Notebooks: `Session2.ipynb` (parts 1–4), `Session2_cont.ipynb` and `Filters_only.ipynb` (part 5). `Filters_only` repeats the code from `Session2_cont` on a different image (`building.jpg`), so both are covered together in part 5.

Order used here

1. Setup and how images live in memory
2. Reading images, video and the camera
3. Color spaces
4. Filtering: smoothing and sharpening
5. Edges: gradients → Sobel family → Laplacian → LoG/DoG → Canny

---

1. Setup and how an image lives in memory

Cell 2: imports and the `show()` helper

```
import cv2 as cv
import matplotlib.pyplot as plt
import numpy as np
import sys
%matplotlib inline
```

- `cv2 as cv` : OpenCV, imported under the short name `cv` . Some notebooks use `cv2` instead; it's the same library.
- `matplotlib.pyplot` : plots images inside the notebook.
- `numpy` : in Python, **every OpenCV image is a NumPy array**.
- `sys` : used later for `sys.exit()` .
- `%matplotlib inline` : a Jupyter "magic" that draws plots under the cell instead of opening a window.

```
def show(img, title=None):
    if img is None: ...
    display_img = cv.cvtColor(img, cv.COLOR_BGR2RGB) if img.ndim == 3 else img
    cmap = None if img.ndim == 3 else 'gray'
```

- If the image failed to load (`None`), it prints a message instead of crashing.
- `img.ndim == 3` means the image is **color** (height × width × 3 channels). It's converted BGR → RGB because matplotlib expects RGB.
- `img.ndim == 2` means **grayscale**, so `cmap='gray'` is used. Without it, matplotlib colors a 2-D array with a yellow/purple palette.
- `plt.axis('off')` hides the pixel rulers.

💡 **What an image is in memory**

Image type	`shape`	`dtype`	Values
Color	`(H, W, 3)`	`uint8`	0–255 per channel, in **B, G, R** order
Gray	`(H, W)`	`uint8`	0 (black) … 255 (white)

- Indexing is **row first**: `img[y, x]`, not `img[x, y]`.
- `uint8` can't go below 0 or above 255. That causes surprises later (see the ⚠️ notes in part 5).

---

2. Reading images, video and the camera

Cells 3–6: reading an image

```
path = './images/dogBGR.jpg'
image = cv.imread(path, -1)
```

- `cv.imread(path, flag)` reads a file into a NumPy array.

Flag	Name	Result
`1`	`cv.IMREAD_COLOR` (default)	3-channel BGR; any transparency is dropped
`0`	`cv.IMREAD_GRAYSCALE`	1-channel gray
`-1`	`cv.IMREAD_UNCHANGED`	Exactly as stored, including the alpha channel

- ⚠️ The markdown only mentions `1` and `0`, but the code uses `-1`.
- ⚠️ If the path is wrong, `imread` **does not raise an error**. It silently returns `None`, and the crash happens later. Always check with `assert image is not None` (`Filters_only` does this).

```
show(image, "reading image")     # correct colors (show converts BGR→RGB)
plt.imshow(image)                # WRONG colors: blue and red swapped
```

- Cell 6 is deliberately wrong. It shows what happens when a BGR array is given to matplotlib, which reads it as RGB.

Cell 7, the hint: OpenCV is BGR and matplotlib is RGB. Remember this; it's the most common beginner bug.

Cells 9–17: opening a video

```
cap_video = cv.VideoCapture('./Videos/video1.mp4')
print(type(cap_video))       # <class 'cv2.VideoCapture'>
print(cap_video.isOpened())  # True
```

- `VideoCapture` opens a **source**: a file path, a camera number (`0`), or a **URL**. 🎯 For the ESP32-CAM you'll pass a stream URL such as `'http://192.168.1.50:81/stream'`.
- `isOpened()` is the only way to know whether it worked.
- Cells 15–17 show that a **wrong path does not raise an error either**. You still get an object, but `isOpened()` is `False`.

Cells 19–25: `get` and `set`

```
cap_video.get(cv.CAP_PROP_FRAME_WIDTH)   # e.g. 1280.0
cap_video.get(cv.CAP_PROP_FRAME_HEIGHT)  # e.g. 720.0
cap_video.get(cv.CAP_PROP_FPS)           # frames per second
cap_video.set(cv.CAP_PROP_FRAME_WIDTH, 320)
```

- `get` returns properties as floats.
- `set` returns `True` or `False` depending on whether the change was accepted.
- ⚠️ For a **video file**, `set(width)` usually returns `False`, because you can't resize a file while reading it; you'd call `cv.resize` on each frame instead. On a **camera**, it asks the driver for that resolution, which works only if the camera supports it.

Cells 27–31: reading one frame

```
retvalue, frame = cap_video.read()
```

- `read()` returns **two things**:
  - `ret` (`True` or `False`): did a frame come out?
  - `frame`: the image (a NumPy array, the same as from `imread`), or `None`.
- `frame.shape` gives something like `(720, 1280, 3)`, which is height, width, channels.

Cell 33: reading all frames

```
while cap_video.isOpened():
    ret, frame = cap_video.read()
    if not ret:
        break
    frame_count += 1
    last_frame = frame
```

- This is the standard loop: read, stop when `ret` is `False` (the end of the video), and otherwise process.
- It prints "Read N **more** frames" because one frame was already consumed in cell 27. A capture is like a tape: it continues from where it stopped.
- ⚠️ After this loop the video is at its end. To read it again, create a new `VideoCapture` or rewind with `cap.set(cv.CAP_PROP_POS_FRAMES, 0)`.

Cell 35: playing video in a window

```
cv.imshow(window_name, frame)
if cv.waitKey(delay) & 0xFF == ord('q'):
    break
...
cv.destroyWindow(window_name)
```

- `cv.imshow` opens a **native OS window**, not a notebook plot.
- `cv.waitKey(delay)` does two jobs:
  1. It waits `delay` milliseconds for a key press and returns the key code, or `-1`.

2. It lets the window **actually draw**. Without `waitKey`, the window stays gray or frozen.

- `& 0xFF` keeps only the lowest 8 bits of the key code, because some systems add extra bits. `ord('q')` is the code for the letter q.
- `delay = 20` gives at most about 50 frames per second. To play at the real speed, use `delay = int(1000 / fps)`.
- ⚠️ This cell never calls `cap.release()`. The camera cell does, and it's a good habit.
- ⚠️ Native windows in Jupyter sometimes freeze. If that happens, run the same code as a `.py` script.

Cell 37: reading from the camera 🎯

```
cap = cv.VideoCapture(0)       # 0 = default webcam
...
cap.release()
```

- This is the same loop as for video, with `0` as the source instead of a file path.
- `delay = 400` means only about 2.5 frames per second are displayed. For smooth live video, use `1`.
- `cap.release()` frees the camera so other programs can use it.
- 🎯 **This is the first building block of the flag detector:** open the camera, read frames in a loop, show them, and press q to quit.

---

3. Color spaces

Cells 39–42: splitting the channels

```
B, G, R = cv.split(image)
```

- `split` returns three separate **2-D** arrays, one per channel. Each one displays as a gray image where **bright means a lot of that color**. For example, a red object looks bright in `R` and dark in `B`.
- The order is **B, G, R**, because that's how OpenCV stores the image.
- Cell 42 does the same with a `for` loop and `zip(("B","G","R"), cv.split(image))`.

Cells 41–46: two ways to get RGB

```
rgb_image = cv.cvtColor(image, cv.COLOR_BGR2RGB)
rgbrev    = image[:, :, ::-1]
```

- `cvtColor(src, code)` is OpenCV's general color-conversion function.
- `[:, :, ::-1]` means "all rows, all columns, channels reversed", so B,G,R becomes R,G,B.
- Both give the same picture.
- ⚠️ The slicing version is a NumPy **view** rather than a real copy, so some OpenCV functions reject it with an error like "Layout of the output array is incompatible". Use `cvtColor` or add `.copy()`.

Cells 48–49: grayscale

```
grayimage = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
```

- The formula is `Gray = 0.299·R + 0.587·G + 0.114·B`. Green gets the most weight because human eyes are most sensitive to it.
- The result has 1 channel, shape `(H, W)`.

Cells 50–54: LAB

- **L** is lightness. **a** runs green ↔ magenta, and **b** runs blue ↔ yellow.
- In 8-bit OpenCV, L is scaled from 0–100 to 0–255, and a and b are shifted so that **128 means neutral** (no color).
- 💡 Color and brightness are separated, so it's good for comparing colors and for color correction.
- ⚠️ `show(LAB_img)` displays strange colors, because `show()` thinks the 3 channels are B,G,R. **Only the split channels (cell 54) are meaningful.** The same applies to YCrCb and HSV.

Cells 55–59: YCrCb

- **Y** is luma (brightness). **Cr** is R − Y, how red the pixel is. **Cb** is B − Y, how blue it is.
- It's used in JPEG compression and in classic skin detection.

Cells 60–64: HSV 🎯 (the most important part for the project)

```
HSV_img = cv.cvtColor(image, cv.COLOR_BGR2HSV)
H, S, V = cv.split(HSV_img)
```

Channel	Meaning	Range in OpenCV (8-bit)
H (hue)	*Which* color: red, yellow, green…	**0–179** (degrees ÷ 2, so it fits in a byte)
S (saturation)	How *pure* the color is (gray → vivid)	0–255
V (value)	How *bright* it is	0–255

💡 **Why HSV for detecting a colored flag**

- In RGB, a shadow on a red flag changes **all three** numbers. In HSV it mostly changes **V**, while **H** stays roughly the same.
- So the plan is to set a **narrow range on H** (the color) and **wide ranges on S and V** (to tolerate lighting).
- Rough hue values in OpenCV: red ≈ 0–10 **and** 170–179, orange ≈ 10–25, yellow ≈ 25–35, green ≈ 35–85, blue ≈ 90–130, purple ≈ 130–160.
- ⚠️ **Red wraps around.** It sits at both ends of the hue range, so you need **two** masks combined: `mask1 | mask2`.
- ⚠️ When S or V is low, the hue becomes meaningless. White, gray and black pixels get random hues, so always set a **minimum S and V**, for example S ≥ 80 and V ≥ 60.

Cell 65: `pair_plot` helper

- Draws two images side by side (`add_subplot(1,2,1)` and `(1,2,2)`).
- It calls `plt.imshow` directly, so **you** must pass RGB. That's why later cells pass `img[:,:,::-1]`.

---

4. Filtering: smoothing and sharpening

Cell 66: the idea of spatial filtering

💡 A **kernel** (also called a mask) is a small grid of numbers, for example 3×3. You slide it over the image, and at each position the output pixel is

`out = Σ (kernel value × pixel under it)`

- Kernel weights that sum to **1** keep the brightness the same.
- Weights that sum to **0** make flat areas black and keep only *changes* (edges).
- Technically OpenCV does *correlation* (it doesn't flip the kernel). For symmetric kernels this is identical to convolution.

Cells 67–68: types of filter

- **Low-pass** (smoothing) keeps slow changes and removes fine detail and noise: mean, Gaussian, median.
- **High-pass** (sharpening and edges) keeps fast changes: Laplacian, sharpen.
- ⚠️ The markdown lists Gaussian as "non-linear". That's wrong: **mean and Gaussian are linear**, and **median is non-linear**, because it sorts the values.

Cells 69–80: the mean (average) filter written by hand

```
AvgKernel = np.array([[1,1,1],[1,1,1],[1,1,1]]) / 9
avg_image = cv.filter2D(img, ddepth=-1, kernel=AvgKernel)
kernel = np.ones((3, 3)) / 9        # same thing, shorter
```

- Each output pixel becomes the average of its 3×3 neighbourhood.
- Dividing by 9 makes the weights sum to 1, so brightness is unchanged.
- `cv.filter2D(src, ddepth, kernel)` applies **any** kernel. You'll reuse it for Sobel, Laplacian and so on.
  - `ddepth=-1` means "the output has the same type as the input" (`uint8`).
  - ⚠️ With `uint8`, results below 0 become 0 and results above 255 become 255 (this is called **saturation**). That's harmless for blurring, but it **loses information for edge kernels** (see part 5).
- Cell 80 calls `pair_plot(img[:,:,::-1], avg_image[:,:,::-1], ...)`, flipping both images to RGB for matplotlib.

Cells 81–84: `cv.blur`, the built-in mean filter

```
out = cv.blur(img, (3, 3))
out = cv.blur(img, (10, 10))   # stronger blur
```

- This is the same as `filter2D` with `ones/9`, just faster and simpler.
- A bigger kernel means a stronger blur. A box filter can have an even size such as 10×10.
- ⚠️ Cell 81 blurs `image` (the dog from earlier), not `img` (the hawk). It's a leftover variable, and the result isn't used.

Cells 85–87: Gaussian blur

```
out = cv.GaussianBlur(img, ksize=(5,5), sigmaX=1)
out = cv.GaussianBlur(img, ksize=(21,21), sigmaX=5)
```

- The weights follow a bell curve: the center counts most, and the weights fade with distance. The result looks **smoother and more natural** than the box filter, without blocky artifacts.
- `ksize` must be **odd and positive**, or `(0,0)` to let OpenCV compute it from sigma.
- `sigma` controls the strength (the width of the bell). A rule of thumb is `ksize ≈ 6σ + 1`.
- The formula in the markdown, `σ = 0.3·((k−1)/2 − 1) + 0.8`, is what OpenCV uses **when you pass `sigmaX=0`**.
- ⚠️ The comment in cell 87 says "sigma=5" for the first call, but the code uses `sigmaX=1`.

Cells 88–93: the median filter, and comparing filters on noise

```
outMed = cv.medianBlur(img, 9)        # ksize is ONE odd int: 3, 5, 7, 9…
```

- It replaces each pixel with the **median** of its neighbours.
- 💡 **Salt-and-pepper noise** is random pure white and black dots. These are extreme outliers, and the median simply ignores them. The mean and Gaussian filters **smear** them into gray blobs instead.
- ⚠️ The comments in cells 90–91 have the wrong sizes: they say 5×5 and 25×25, but the code uses 3×3.

Cell 93, the full experiment:

```
from skimage.util import random_noise
noisyImg = random_noise(img, mode="s&p", amount=0.02)   # returns float 0..1
noisyImg = np.array(255*noisyImg, dtype="uint8")        # back to 0..255 uint8
```

- Noise is added to 2% of the pixels, then all three filters are compared.
- **Lesson: the type of noise decides the filter.** Use median for salt-and-pepper, and Gaussian for grainy sensor noise.
- `skimage` (scikit-image) has to be installed separately.

Cells 94–100: high-pass filters

```
laplacian_filt = [[0, 1, 0],
                  [1,-4, 1],
                  [0, 1, 0]]        # sums to 0 → flat areas become 0 (black)
sharpen_filter = [[0,-1, 0],
                  [-1, 5,-1],
                  [0,-1, 0]]        # sums to 1 → brightness kept, edges boosted
```

- **Laplacian:** in a flat area the center (−4) cancels the four neighbours (+1 each), giving 0. At an edge they don't cancel, so the pixel lights up. The result is a mostly black image with bright edges.
- **Sharpen** = identity − Laplacian. It's the original image plus extra edge contrast. The moon (cell 100) shows the craters popping out.

Cell 101: exercises

Exercise	How
Camera feed → gray	`cv.cvtColor(frame, cv.COLOR_BGR2GRAY)` inside the camera loop
Sharpen each frame	`cv.filter2D(frame, -1, sharpen_filter)`
Add salt-and-pepper noise	`random_noise(frame, mode="s&p", amount=0.02)`, then ×255 and convert to `uint8`
Remove it	`cv.medianBlur(noisy, 3)`

---

5. Edges (`Session2_cont.ipynb` and `Filters_only.ipynb`)

Cell 1: loading

```
img = cv2.imread("images/cameraman.tif", cv2.IMREAD_GRAYSCALE)
```

- Edge detection works on **one channel**, so the image is loaded as grayscale.

Cells 2–3: gradients with Sobel

💡 Treat the image as a height map `I(x, y)`. The **gradient** measures how fast brightness changes:

- `∂I/∂x`, the change going **left** → **right**, is large on **vertical** edges.
- `∂I/∂y`, the change going **top** → **bottom**, is large on **horizontal** edges.
- **Magnitude** `√(gx² + gy²)` is the edge strength in any direction.
- **Orientation** `atan2(gy, gx)` is the direction the brightness increases.

```
grad_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)   # dx=1, dy=0
grad_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)   # dx=0, dy=1
magnitude   = np.sqrt(grad_x**2 + grad_y**2)
orientation = np.arctan2(grad_y, grad_x) * 180 / np.pi
```

- `cv2.CV_64F` makes the output **64-bit float**, so **negative values survive**. A dark→bright edge is positive and a bright→dark edge is negative. With `uint8`, every negative would become 0, and you'd lose half the edges.
- `orientation` is computed but never plotted.
- ⚠️ The plot titles are **swapped**: "Gradient X (Horizontal edges)" should say *vertical* edges. The markdown above it is correct.

Cells 5–7: Laplacian (second derivative)

```
laplacian = cv2.Laplacian(img, cv2.CV_64F)
```

- This is the rate of change of the rate of change. Edges sit where it crosses zero (**zero-crossings**).
- It gives no direction, and it's **very sensitive to noise**, which is why LoG (below) blurs first.

Cells 10–11: Roberts, Prewitt and Sobel kernels

Filter	Size	Notes
Roberts	2×2	Diagonal differences; tiny, so very noisy
Prewitt	3×3	`[-1,0,1]` in each row
Sobel	3×3	`[-1,0,1],[-2,0,2],[-1,0,1]`: the center row counts double, which adds a little smoothing

```
sobel_3x3x = cv2.getDerivKernels(1, 0, 3)[0]
rob_x = cv2.filter2D(img, -1, roberts_x)
```

- `getDerivKernels` returns Sobel as two **1-D** pieces (Sobel is "separable"). It's computed here but never used.
- ⚠️ `filter2D(..., -1, ...)` on a `uint8` image clips negatives to 0, so you only see edges of **one polarity**. To see all of them, use `cv2.CV_64F` and take `np.abs(...)`, or use `cv2.convertScaleAbs(...)`.

Cells 12–13: Sobel 3×3 vs 5×5

- **3×3** gives sharper, thinner edges but more noise.
- **5×5** gives smoother, thicker edges that are less noisy.
- This trade-off between detail and robustness is everywhere in computer vision.

Cells 14–16: LoG and DoG

```
blur = cv2.GaussianBlur(img, (5,5), 1)
log  = cv2.Laplacian(blur, cv2.CV_64F)          # LoG = Laplacian of Gaussian
blur1 = cv2.GaussianBlur(img, (5,5), 1)
blur2 = cv2.GaussianBlur(img, (5,5), 2)
dog = blur1 - blur2                               # DoG = Difference of Gaussians
```

- **LoG:** blur first to kill noise, then apply the Laplacian.
- **DoG:** subtract two blurs of different strength. It approximates LoG and is cheaper. **SIFT** uses DoG to find blobs (Session 3).
- ⚠️ **Important bug:** `blur1 - blur2` is computed on `uint8`, so negative results **wrap around**: `10 - 20 = 246`, not −10. The DoG image is therefore full of false bright speckles. The fix is to compute in float: `dog = blur1.astype(np.float32) - blur2.astype(np.float32)`

Cells 8–9: Canny, which uses everything above

```
edges = cv2.Canny(img, 100, 200)
```

Canny runs five stages:

1. **Smooth** the image to reduce noise. ⚠️ OpenCV's `Canny` does **not** blur for you, so call `GaussianBlur` first yourself (Session 5 does this).
2. Compute the **gradient** with Sobel: magnitude and direction.

3. **Non-maximum suppression:** along the gradient direction, keep only the peak pixel. This thins edges to one pixel wide. The direction is rounded to 0°, 45°, 90° or 135° (that's the semicircle picture).
4. **Double threshold:** gradient > 200 is a *strong* edge; < 100 is discarded; anything between is *weak*.
5. **Hysteresis:** keep weak pixels only if they **touch** a strong edge.

- The output is a binary `uint8` image: 255 for edges and 0 everywhere else.
- A low:high ratio of 1:2 or 1:3 is the usual rule of thumb.

`Filters_only.ipynb`

- It's the same code on `building.jpg`. Buildings have lots of straight vertical and horizontal lines, so the difference between X and Y gradients is very clear there.
- It adds `assert img is not None, "Image not found"`, a good habit to copy.

Session 2 gotchas, all in one place

1. BGR vs RGB: use `cv.cvtColor` before `plt.imshow`.
2. `imread` and `VideoCapture` fail **silently**, so always check for `None` and `isOpened()`.
3. Video `set(width)` usually fails on files; resize frames instead.
4. `waitKey` is required for `imshow` windows to update.
5. Composite LAB, YCrCb and HSV images look meaningless; look at the split channels instead.
6. OpenCV hue is **0–179**, and red wraps around.
7. `uint8` math clips (in filters) or wraps (in NumPy `-`). Use `CV_64F` or `float32` for edges and differences.
8. Gaussian blur is linear, not non-linear.
9. Sobel-X finds **vertical** edges (the plot title is wrong).

🎯 What from this session goes into the flag detector

- The camera loop (cell 37), with the stream URL in place of `0` later.
- `cvtColor(frame, COLOR_BGR2HSV)` (cell 62).
- A light `GaussianBlur` before thresholding to reduce noise flicker.
- Understanding H/S/V ranges, including red's two hue ranges.